Processamento de Transações

Laboratório

robespierre.pita@ufba.br

robespierre.pita@ufba.br

O problema

BEGIN / COMMIT / ROLLBACK

No PSQL Tool (sessão 1)	No Query Tool (sessão 2)
Crie uma transação que adicione 1000 reais ao salário de João na tabela funcionário. BEGIN; QUERY; COMMIT; ⚠	Antes do “COMMIT;” na sessão 1… Crie uma query que consulte o salário atual de João.

```
SELECT pg_current_wal_lsn();
```

robespierre.pita@ufba.br

0 problema

BEGIN / COMMIT / ROLLBACK

No PSQL Tool (sessão 1)

```
SELECT pg_current_wal_lsn();

BEGIN;
update funcionario
set salario = salario + 1000
where primeiro_nome = 'João';
COMMIT;

SELECT pg_current_wal_lsn();
```

No Query Tool (sessão 2)

```
SELECT Primeiro_nome, Salario
FROM FUNCIONARIO
WHERE Primeiro_nome = 'João';
```

isolamento na prática - testa o “ROLLBACK;” agora

robespierre.pita@ufba.br

O problema

lock de linha

No PSQL Tool (sessão 1)
Crie uma transação que adicione **500** reais ao salário do/a funcionário/a com CPF ‘12345678966’. BEGIN; QUERY; COMMIT; 

No Query Tool (sessão 2)
Crie uma transação que adicione **300** reais ao salário do/a funcionário/a com CPF ‘12345678966’. BEGIN; QUERY; COMMIT; 

```
SELECT pg_current_wal_lsn();
```

robespierre.pita@ufba.br

O problema

lock de linha

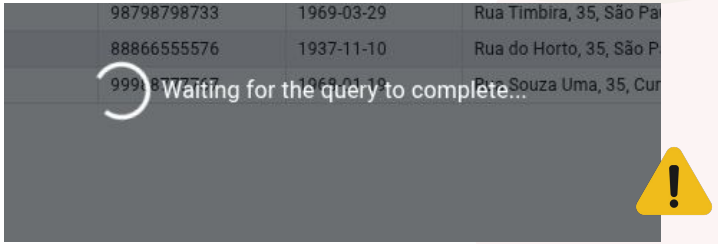
No PSQL Tool (sessão 1)

```
BEGIN;

UPDATE FUNCIONARIO
SET Salario = Salario + 500
WHERE Cpf = '12345678966';
```

No Query Tool (sessão 2)

```
UPDATE FUNCIONARIO
SET Salario = Salario + 300
WHERE Cpf = '12345678966';
```



COMMIT;

O problema

lock de linha

No PSQL Tool (sessão 3)

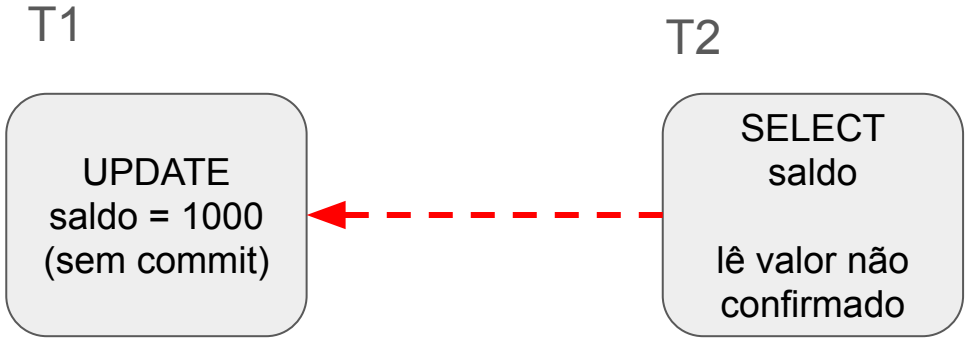
```
SELECT pid, state, query
FROM pg_stat_activity
WHERE datname = current_database();
```

robespierre.pita@ufba.br

Aprofundando no isolamento de transações

Fenômenos indesejáveis entre transações concorrentes

Leituras sujas (dirty reads)



Uma transação lê dados escritos por outra transação concorrente que ainda não foi confirmada.

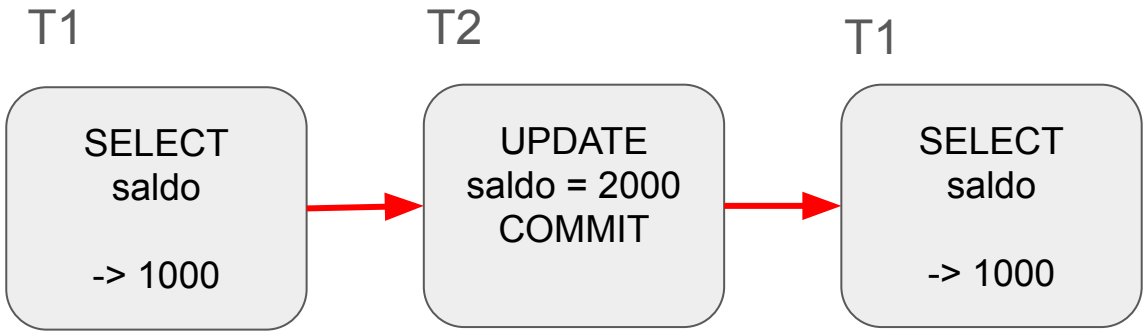
Fonte: https://www.postgresql.org/docs/7.1/transaction-iso.html

robespierre.pita@ufba.br

Aprofundando no isolamento de transações

Fenômenos indesejáveis entre transações concorrentes

Leituras não repetíveis (non-repeatable reads)



Uma transação relê dados que havia lido anteriormente e descobre que eles foram alterados por outra transação (que realizou commit).

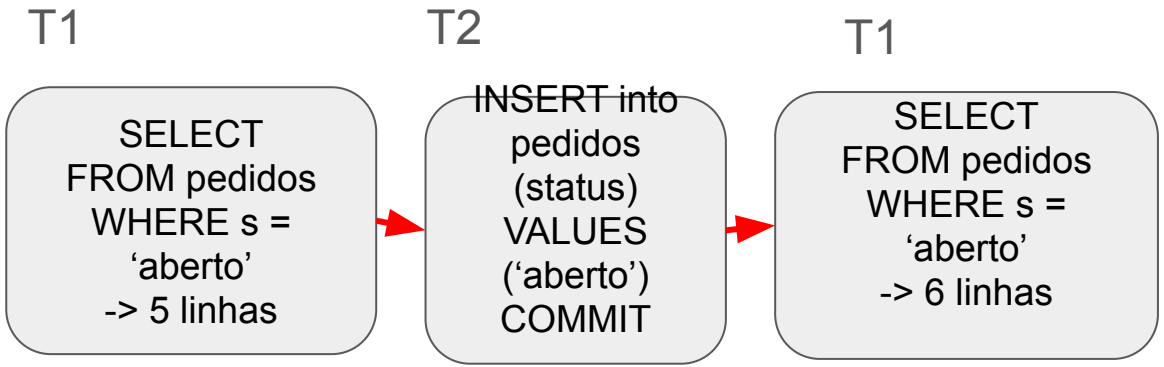
Fonte: https://www.postgresql.org/docs/7.1/transaction-iso.html

robespierre.pita@ufba.br

Aprofundando no isolamento de transações

Fenômenos indesejáveis entre transações concorrentes

Leituras fantasmas (phantom reads)



Uma transação re-executa uma consulta que retorna um conjunto de linhas que satisfazem uma condição e descobre que o conjunto de linhas mudou devido outra transação que realizou commit.

Fonte: https://www.postgresql.org/docs/7.1/transaction-iso.html

Aprofundando no isolamento de transações

Fenômenos indesejáveis entre transações concorrentes

Table 9-1. ANSI/ISO SQL Isolation Levels

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read
Read uncommitted	Possible	Possible	Possible
Read committed	Not possible	Possible	Possible
Repeatable read	Not possible	Not possible	Possible
Serializable	Not possible	Not possible	Not possible

Postgres offers the read committed and serializable isolation levels.

Fonte: https://www.postgresql.org/docs/7.1/transaction-iso.html

robespierre.pita@ufba.br

READ COMMITTED

Cada comando SELECT enxerga apenas dados que já foram commitados no momento em que o comando começou.

No PSQL Tool (sessão 1)	No Query Tool (sessão 2)
Crie uma transação que adicione **5000** reais ao salário do/a funcionário/a com CPF ‘66688444476’. BEGIN; QUERY; COMMIT;	Consulte o salario do funcionário com cpf ‘66688444476’

robespierre.pita@ufba.br

READ COMMITTED

Cada comando SELECT enxerga apenas dados que já foram commitados no momento em que o comando começou.

No PSQL Tool (sessão 1)

```
BEGIN;

UPDATE FUNCIONARIO
SET salario = 5000
WHERE cpf = '66688444476';
```

No Query Tool (sessão 2)

```
SELECT salario
FROM FUNCIONARIO
WHERE cpf = '66688444476';
```

robespierre.pita@ufba.br

READ COMMITTED

Cada comando SELECT enxerga apenas dados que já foram commitados no momento em que o comando começou.

No PSQL Tool (sessão 1)
Crie uma transação que selecione o salario de João

No Query Tool (sessão 2)
Crie uma query que altere adicione 2000 ao salário de João, finalize com COMMIT;

robespierre.pita@ufba.br

READ COMMITTED

Cada comando SELECT enxerga apenas dados que já foram commitados no momento em que o comando começou.

No PSQL Tool (sessão 1)

```
BEGIN;

SELECT Salario
FROM FUNCIONARIO
WHERE Primeiro_nome = 'João';
```

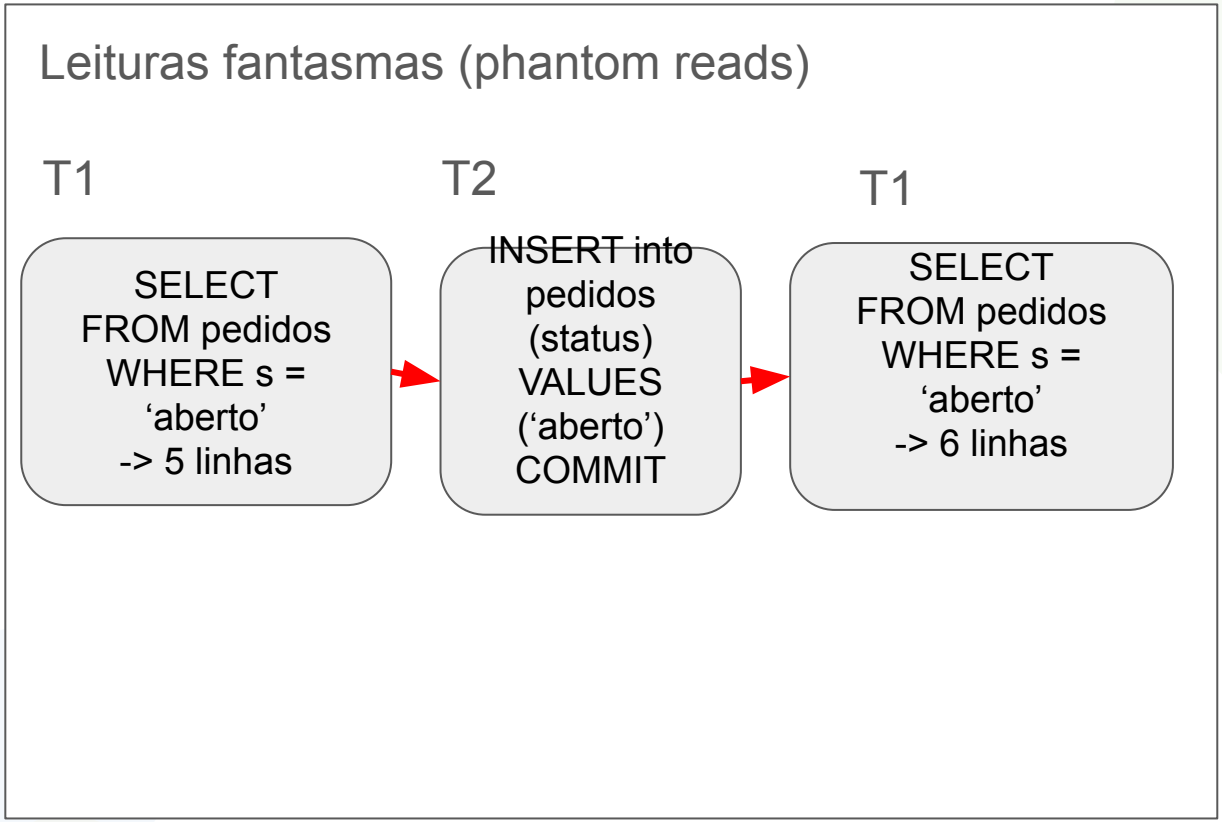
No Query Tool (sessão 2)

```
UPDATE FUNCIONARIO
SET Salario = Salario + 2000
WHERE Primeiro_nome = 'João';

COMMIT;
```

robespierre.pita@ufba.br

Elabore um exemplo de Phantom Read



Obrigado!!

robespierre.pita@ufba.br